



FieldsFuzz: Implement efficient fuzzing based on grammar-aware mutation strategy[☆]

Yinghao Su^{a, *}, Dapeng Xiong^{b, *}, Kechang Qian^b, Yu Wang^b, Qingyao Zeng^a

^a Institute of Graduate, Space Engineering University, Beijing 101416, China

^b Institute of Aerospace Information, Space Engineering University, Beijing 101416, China

ARTICLE INFO

Keywords:

Fuzzing
Taint analysis
Structure-aware
Seed mutation

ABSTRACT

A comprehensive understanding of the input format utilized by the testing program is essential for the generation of valid inputs and the enhancement of testing efficacy in fuzz testing. Nevertheless, current format-aware fuzz testing tools predominantly focus on recognizing various functional segments of binary input files, usually overlooking the structural intricacies and dependencies inherent within these files. Furthermore, existing format-aware methodologies that based on comparison and taint analysis exhibit limitations in accurately identifying file fields and types. To mitigate these challenges, this article introduces a novel format-aware fuzz testing tool, termed FieldsFuzz. Initially, FieldsFuzz performs byte-level taint analysis on significant seed inputs during program execution to derive a set of input byte instructions, thereby identifying input file structures and dependencies, and constructing a file format tree. During the mutation phase, FieldsFuzz traverses the file format tree to ascertain field dependencies, executes fields and dependencies based mutations to enhance the efficiency of effective seed generation, and introduces random modifications to the file structure to uncover previously unknown vulnerabilities. An evaluation of FieldsFuzz was conducted using twelve distinct format input programs, revealing that it surpasses leading fuzzing tools (including AFL, AFL++, WEIZZ, ProFuzzer, and NestFuzz) in terms of format recognition accuracy, code coverage, and the detection of security vulnerabilities.

1. Introduction

Fuzz testing, a dynamic software testing method, has proven highly effective in identifying and exposing unknown security vulnerabilities. Typically, fuzz testing tools generate testcases based on predefined strategies to conduct thorough testing of target applications, uncovering potential security weaknesses by monitoring anomalies during program execution (Godefroid, 2020). The key to enhancing the efficiency of fuzz testing lies in generating input in an effective format (Beaman et al., 2022). A standard input case usually consists of a sequence of data fields with specific semantics, such as buffer size and checksum. The program interprets these fields according to its implemented functionality, and invalid input cases may be rejected early in the program's execution, which complicates efforts to improve the coverage of program testing.

The currently available fuzz testing tools primarily generate test cases through random mutation and predefined rules. However, mutation-based fuzz testing tools direct mutate seeds to create testcases based on edge coverage. Fuzz generators often spend excessive time

producing numerous invalid inputs, which are typically rejected during the initial parsing stage of certain programs, leading to minimal or no enhancement in code coverage. In contrast, generation-based fuzz testing tools create test cases using predefined seed formats, which somewhat mitigates the generation of invalid inputs (Zhu et al., 2022). Nevertheless, this approach depends on prior knowledge and may not accurately represent the effective range of program input file parsing.

In recent years, many fuzz testing tools have developed file format awareness by collecting feedback from the fuzz testing process, thereby enhancing the effectiveness of generating testcases. ProFuzzer (You et al., 2019) and WEIZZ (Fioraldi et al., 2020a) employ a byte-by-byte mutation method to identify file fields. However, determining field boundaries through byte flipping necessitates a substantial number of inputs, resulting in low accuracy and efficiency in identifying these fields. AIFORE (Shi et al., 2023) utilizes neural network models to learn the boundaries and types of various fields but lacks a mutation strategy for recognizing and targeting regional relationships. NestFuzz (Deng et al., 2023) employs an input processing tree to represent the input

[☆] Editor: Burak Turhan.

^{*} Corresponding author.

E-mail address: xiongdapeng@hgd.edu.cn (D. Xiong).

structure; however, this tree is constructed based on program instructions, leading to a generated tree structure with duplicate fields and excessive depth, which compromises the accuracy of identifying the input file structure and its dependency relationships.

Based on the analysis above, existing format-aware fuzz testing tools primarily focus on identifying fields, but they lack dependence on input fields and structural recognition. As a result, the generated test cases often fail to meet program requirements. These tools are ineffective in exploring complex real-world programs and are unable to trigger deep vulnerabilities. To address these challenges, we propose FieldsFuzz, a format-aware fuzz testing tool. To accurately identify the input file format, FieldsFuzz employs a recognition method based on taint analysis. This tool utilizes taint analysis technology to generate a list of taint instructions present in the input file during program execution. The taint instruction data is then simplified, sorted, and classified. Subsequently, the input file is divided into fields, and a file format tree is constructed according to these field boundaries. FieldsFuzz matches each field's taint list with predefined field types and rule sets to determine the type of each file field. Simultaneously, for each field type, it records the dependency relationships and special values between that field and other fields based on the taint instruction list, adding this information to the dependency relationship table. To enhance mutation efficiency, FieldsFuzz has implemented a file format-based mutation method. This approach is divided into two stages: field mutation and structural mutation. In the field mutation stage, FieldsFuzz traverses the file format tree to identify file fields and their dependency relationships. It then designs mutation rules based on the types of fields and dynamically adjusts the dependency relationships among these fields during the mutation process. This ensures the integrity of the input structure while exploring deeper paths within the program, significantly improving the effectiveness of test case generation. In the structural mutation stage, FieldsFuzz modifies the structure of the file format tree by randomly adding or removing nodes, thereby investigating program vulnerabilities that may arise from file structure errors.

We implemented FieldsFuzz based on Pin (Luk et al., 2005) and AFL (Zalewski, 2023) and evaluated its effectiveness using benchmarks from 12 rigorously tested real-world programs. The benchmark testing program includes 10 common input file formats, such as binary files, images, videos, and audio formats. We compared FieldsFuzz with advanced structure-aware fuzzers (including WEIZZ (Fioraldi et al., 2020a), ProFuzzer (You et al., 2019), and NestFuzz (Deng et al., 2023)) and general-purpose fuzzers (including AFL and AFL++ (Fioraldi et al., 2020b)) using the aforementioned benchmark test program. The results indicated that, compared to AFL, AFL++, ProFuzzer, WEIZZ, and NestFuzz, FieldsFuzz increased edge coverage by 30.30%, 11.88%, 21.53%, 15.50%, and 5.95%, respectively, and enhanced the vulnerability discovery count by 139.88%, 27.30%, 46.64%, 28.48%, and 10.96%, respectively.

In summary, the key contributions of our paper are outlined as follows.

- Propose a method for recognizing the boundaries and types of input file fields based on taint analysis, which can effectively identify these fields and their associated relationships.
- A method for saving input structures using file format trees and dependency tables has been suggested.
- A file format-based mutation method is proposed, utilizing the file structure tree. This method incorporates input field correlation during the file mutation process to increase the proportion of effective test cases generated and enhance the efficiency of fuzz testing.
- We have developed a format-aware fuzz testing tool, FieldsFuzz, and systematically evaluated it across a diverse array of input formats and programs. The results demonstrate that our algorithm surpasses other advanced fuzz testing tools in both input format recognition accuracy and fuzz testing performance.

```

1  typedef struct {
2      long length;   char magic;
3      int width;    int height;
4      int* data;    int checksum;
5  } FieldFile;
6  unsigned int calculate_checksum(const int* data, int
length);
7  void read_field_data(int* data, int length, FILE* fd);
8  FieldFile* read_field_file(const char* filename) {
9      FILE* fd = fopen(filename, "rb");
10     FieldFile* field = malloc(sizeof(FieldFile));
11     if (!field)
12         exit(0);
13     fread(&field->magic, sizeof(char), 1, fd);
14     if (field->magic != 'a' )
15         exit(0);
16     fread(&field->length, sizeof(int), 1, fd);
17     fread(&field->checksum, sizeof(int), 1, fd);
18     fread(&field->width, sizeof(int), 1, fd);
19     fread(&field->height, sizeof(int), 1, fd);
20     int data_size = field->width * field->height;
21     if (data_size != field->length)
22         exit(0);
23     field->data = malloc(data_size * sizeof(int));
24     if (!field->data)
25         exit(0);
26     read_field_data(field->data, data_size, fd);
27     int calculated_checksum = calculate_checksum
(field->data, data_size);
28     if (calculated_checksum != field->checksum)
29         exit(0);
30     fclose(fd);
31     return field;
32 }

```

Fig. 1. Motivation example.

2. Motivation

As an example of motivation, Fig. 1 illustrates an input format parsing program. This program reads files and verifies their checksums. The code defines a structure called 'FieldFile', which consists of several specific file fields, including input length, magic byte, height, width, valid data, and checksum. Each field has its own unique function and meaning. When the format parsing program reads FieldFile, it first reads and verifies the magic byte, and then reads the input length, height, and width to verify the relationship between file size and height width. The parsing program reads data according to file size, calculates the checksum, and verifies whether the input checksum field is the same. In the above process, if a verification error occurs, the program will immediately exit. Fuzz testing tools based on mutation usually use random mutation to generate testcases, which makes it difficult to ensure input case integrity. The verification process of reading and parsing programs for mutation testcases often fails, making it difficult to improve program path coverage. At the same time, mutation based testing tools uniformly mutate bytes without understanding the input field function, which requires a long time to trigger vulnerabilities in random mutations. For example, there is an integer overflow vulnerability in program line 20 in Fig. 1, which requires targeted modification of width and height to trigger the vulnerability. Mutation based testing tools are difficult to trigger this vulnerability.

The current format-aware fuzz testing tool also has limitations when testing this program. ProFuzzer employs byte flipping and statistical analysis methods to identify types of input fields. During the fields recognition process, ProFuzzer provides feedback on coverage similarity, which results in limitations in recognizing field types. For instance, in Fig. 1, lines 14 and 28 of the program verify the input Magic byte and Checksum fields, respectively. These fields must meet specific values to trigger coverage changes, yet ProFuzzer cannot distinguish

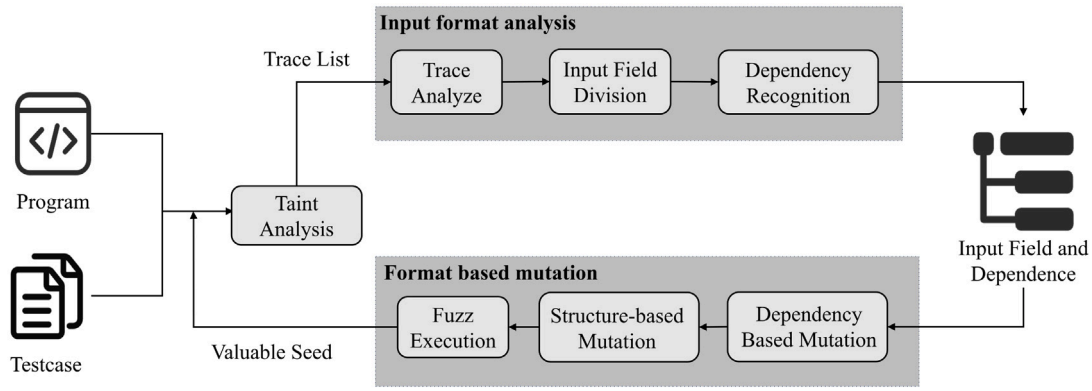


Fig. 2. The overall architecture of FieldsFuzz.

between them. Existing format-aware fuzz testing tools primarily focus on segmenting fields based on regional functions while neglecting the relationships between these fields. For example, in the program parsing file shown in Fig. 1, when the fuzz testing tool increases the length of the data segment without altering the input length value, the program accepts the data field input according to the original size, discards the additional portion, and diminishes the effectiveness of seed mutation.

Based on the analysis presented above, we have reached the following conclusion: Identifying input file formats can enhance the efficiency of fuzz testing. Additionally, employing different mutation methods for various fields of the file can expedite the exploration and discovery of unknown vulnerabilities. There exists a dependency relationship among different fields of the input file. To achieve legitimate mutations, it is essential to consider these field dependencies during the mutation process, ensuring that the program can parse the input correctly and that the test cases can explore deeper paths within the program. Due to the complex structure and variable states of the testing program, format-aware fuzz testing tools based on byte flipping often have difficulty accurately identifying file formats. In addition, although the byte flipping format aware strategy is faster in single input execution rate than the taint tracking file format aware strategy, flipping a single input file byte by byte also requires a lot of time, and compared to taint analysis methods, it also has shortcomings in format aware efficiency. Based on the above issues, this article proposes a FieldsFuzz file format-aware fuzz testing tool.

3. Design

The FieldsFuzz architecture is illustrated in Fig. 2. FieldsFuzz divides the fuzz testing process into two stages: format analysis and format-based mutation. First, FieldsFuzz selects seeds from the seed queue to input into the test program and performs instruction-level taint analysis on the target program to generate a list of taint instructions corresponding to the input bytes. Subsequently, FieldsFuzz analyzes the taint instructions and segments the input file into byte-based fields. By examining the relationship between instruction control flow and data flow, the field types and dependencies are identified, leading to the construction of a file format tree and a dependency table. During the mutation phase, FieldsFuzz modifies seed files according to the file format tree. In the dependency mutation stage, FieldsFuzz alters the nodes of the file format tree while capturing the dependency relationships of those nodes. Utilizing these relationships, relevant fields are adjusted to create valid test cases, thereby preventing fuzz testing tools from generating a large number of invalid test cases. This approach ensures the validity of the test cases and enhances testing efficiency. Additionally, destructive mutations of the input seed file format tree are performed during the structural mutation stage to uncover unknown vulnerabilities that may be triggered by malformed format files.

3.1. Input format analysis

As mentioned earlier, the format of input file fields can provide significant guidance for fuzz testing, and different fields can employ various mutation methods in this process. Therefore, identifying the types of input file fields is crucial for enhancing the effectiveness of program fuzz testing. Fig. 3 illustrates the fundamental process of recognizing file input formats. In this process, the FieldsFuzz class classifies and organizes the list of corrupted instructions based on the starting position and length of the corrupted byte of the operand, constructing an input format tree according to the field boundaries. Simultaneously, FieldsFuzz identifies the type of field and its dependency relationships with other fields by analyzing the characteristics and associated fields of each field's taint instruction, ultimately creating a field dependency table.

To illustrate the input format recognition method, we define the commonly used terms in the subsequent content as follows:

Definition 1. The file field is defined by a series of consecutive bytes in the input file, which includes the initial byte position, field length, and field type. This can be represented as $F = \{byte_start, length, type\}$. When the field type is not considered, it can be abbreviated as $F(byte_start, length)$.

Definition 2. An input format tree is a detailed representation method of an input file's structure. The input format tree adheres to the general definition of a tree, with its nodes comprising a list of pointers to file fields and subfields, represented as $Node = \{F, Sub_list\}$.

Definition 3. A field dependency table is a record that illustrates the relationships between input fields. This table documents the dependency fields or valid value sets associated with each field. The field dependency table record is expressed as $List_item = \{F_key, Value_set\}$, where F_key represents the key value of the field, and $Value_set$ denotes the set of valid values. $Value_set$ records the corresponding dependency field or dependency value set for field F .

3.1.1. Dynamic taint analysis

The dynamic taint analysis engine employs input test cases provided by coverage-based fuzzers to monitor contamination at the instruction level of the target program and to obtain the execution trajectory of these test cases. In this process, the DTA engine utilizes testcases as taint sources, bytes as the granularity for taint propagation, and selects bytes as the fundamental unit for taint propagation to satisfy both accuracy and performance requirements. By determining which memory locations and registers depend on contaminated inputs during program execution, byte-granularity taint analysis can trace contaminated values back to a specific offset in the input case. In order to obtain the byte behavior characteristics of test cases, the taint tracing engine tracks the

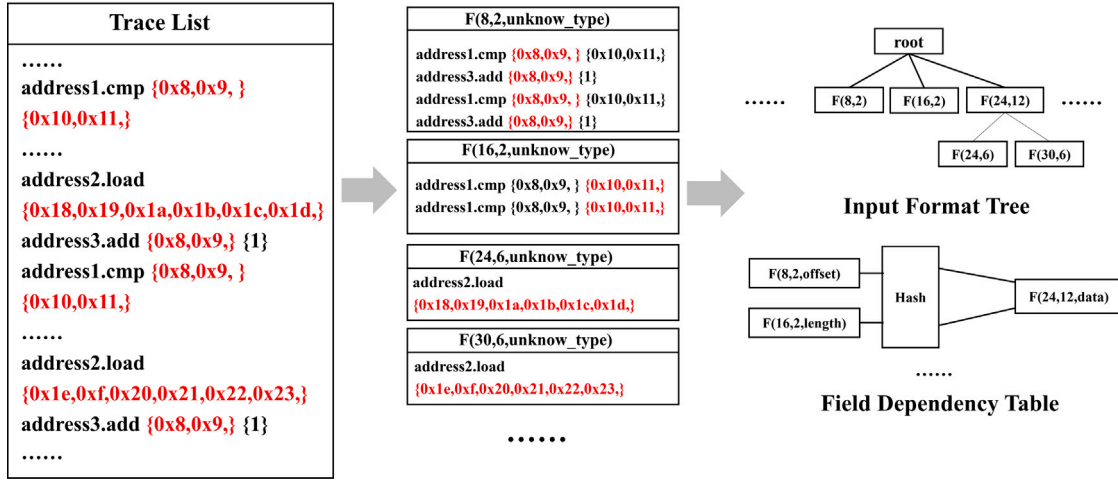


Fig. 3. The Process of input format analysis. FieldsFuzz generates (a) Taint byte analysis list based on program execution and taint analysis, and generates (b) Fields divided trace list through instruction list analysis and partitioning. Generate (c) input format tree and (d) Fields dependency table based on field recognition and dependency recognition.

execution instructions of the target program, records program jumps and numerical operation instructions such as cmp, load, je, etc., and generates an execution trace list. The execution tracking list consists of taint tracking instructions, which include all instructions related to taint variables during program execution. The taint tracking instruction is defined as a quadruple $Taint_trace = \{address, opcode, var1, var2\}$, where

The *address* indicates the location of the instruction, while the *opcode* specifies the type of operation being performed. The variables *var1* and *var2* represent the two operands of the instruction. Each operand is further defined as a structure $var = \{byte_field, value\}$. Here, *byte_field* records the byte offset in the input file corresponding to the tainted operand, and *value* captures the specific value of the constant variable.

Due to the presence of loop structures and single-byte comparison statements in the code, there are numerous redundant entries in the tracking instruction list. Additionally, many instructions with the same address are executed for the same field data, resulting in the generation of multiple taint tracking instructions. This redundancy negatively impacts the efficiency of field recognition.

FieldsFuzz consolidates redundant records by analyzing the characteristics of the tracking instruction list and optimizing the record format. The tracking records are merged according to the following rules:

- Merge the variable offset fields of consecutive instructions located at the same address and create a single merged instruction.
- Instructions with adjacent addresses can be merged if the operator is the same and the taint field of an operand is identical. In this case, merge the two instructions and combine the taint fields of the different operands.
- Due to the repeated instructions generated by program loop statements, only one instruction is retained.

Based on the aforementioned rules, FieldsFuzz integrates detailed operational instructions, streamlines the instruction analysis list, and decreases the computational cost of field recognition. Fig. 4 illustrates an example of merging redundant records. In the program depicted in Fig. 4(a), a while loop is executed, during which the program performs a byte read instruction followed by a comparison instruction until the loop terminates. Based on taint analysis, the instruction trace list shown in Fig. 4(b) was generated. To reduce redundancy in the instruction trace list, FieldsFuzz merges redundant records into a single instruction, resulting in a simplified trace list as presented in Fig. 4(c). In Fig. 4(b), the red and green instructions exhibiting similar behavior are consolidated into the red and green instructions in Fig. 4(c).

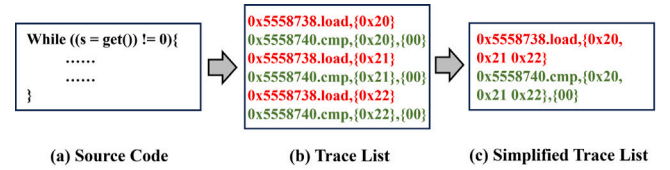


Fig. 4. An example of simplified trace list.

3.1.2. Fields recognition and dependency inference

Based on the simplified instruction analysis list, FieldsFuzz categorizes the bytes of the input file according to their respective functions and groups fields with similar functions into designated fields of the file. This approach enables targeted mutation of the seed file. The process of field recognition primarily consists of two components: field boundary division and field type identification.

Identifying the boundaries of different fields is essential for segmenting the input file field. Due to the phenomenon of overlapping effective fields in instructions, it is often inappropriate to divide fields at the instruction level. For instance, an instruction to read bytes may parse multiple fields, leading to the erroneous merging of different fields or the subdivision of fields of the same type. To address this problem, FieldsFuzz segments the input file into fields according to a simplified instruction list. It preliminarily identifies the field boundaries, constructs an input format tree, determines the input structure, and merges adjacent fields with similar behaviors to enhance the accuracy of field division.

FieldsFuzz first processes the taint instructions in the simplified instruction analysis list by sorting and classifying them according to the initial byte and offset of the tainted area associated with the operand. Due to the merging of instruction action areas with similar behavior during the process of simplifying instructions, excessive refinement of field partitioning is effectively avoided. Next, FieldsFuzz divides the consecutive tainted bytes in the tainted instruction operand into a single field. For operands that encompass discontinuous tainted bytes, they are separated into multiple independent fields to ensure that each field contains only one continuous set of tainted bytes. Through these steps, FieldsFuzz achieves preliminary field partitioning of input files based on tainted instructions.

FieldsFuzz determines the nested relationships between adjacent fields by constructing an input format tree. Algorithm 1 describes the process for creating a file format tree. Firstly, FieldsFuzz initializes the file format tree, with the root node representing the entire seed

file field. It then inserts other fields into the file format tree based on their starting bytes and byte lengths. During the insertion of a node, FieldsFuzz recursively searches for the deepest node that encompasses the interval, using it as the parent node according to the insertion node's interval. Subsequently, it calculates the taint similarity between the instruction list of this node (Cui et al., 2016) and the neighboring interval nodes. If the taint matching degree exceeds a specified threshold, the field is merged with neighboring field nodes to prevent fields of the same type from being divided into separate nodes. Utilizing Algorithm 1, FieldsFuzz accurately categorizes the input file into a more precise file format tree based on the types of fields.

Algorithm 1 Build Format Struct Tree

```

1: struct Node { Byte_begin, Length, Intervals, Sub_list }
2: struct Tree
3: function INITIALIZE
4:   Root ← New_Node(0, length(input_file), [])
5: end function
6: function INSERT_NODE(Cur_Node, Search_Node)
7:   if ¬(Cur_Node.Intervals ∈ Search_Node.Sub_list.Intervals) then
8:     Find_NeighborNode(Search_Node)
9:     if Similarity(NeighborNode, Cur_Node) then
10:      Merge_Node(NeighborNode, Cur_Node)
11:   else
12:     Search_Node.Sub_list.insert(Cur_Node)
13:   end if
14:   return
15: end if
16:   Contain_Node ← FindContainNodes(Cur_Node,
    Search_Node.Sub_list)
17:   Insert_Node(Cur_Node, Contain_Node)
18: end function
19: function BUILD_TREE
20:   Field_List ← Get_Field(Taint_List)
21:   Sort(Field_List)
22:   while not Field_List.IsEmpty() do
23:     Cur_Node ← (Field_List.pop(), [])
24:     Insert_Node(Cur_Node, Root)
25:   end while
26: end function

```

The type of file field directly impacts the ability and efficiency of generating effective testcases during the mutation process in fuzz testing. Based on common file format standards and functions, FieldsFuzz has established the primary categories for identifying effective file field types, as illustrated in Table 1. There are different rules in the number of instructions recorded in different types of fields and the taint bytes relative to instructions. In the field type recognition stage, FieldsFuzz initializes a set of field rules that includes several common rule features. For example, the length field is typically associated with other data fields, and the corresponding instruction list contains a significant number of consecutive *cmp* or *load* instructions. During the type recognition phase, FieldsFuzz enumerates the fields of all nodes in the file format tree, reads the taint instruction list for each field, extracts instruction features, and identifies the corresponding field type based on the taint instruction feature rules associated with different fields. In the recognition of Magic numbers and Enumeration fields, since these taint instructions are typically compared with valid values, FieldsFuzz records the corresponding valid value set after identifying these fields.

The purpose of dependency inference is to analyze the effective scope of the taint instruction list within a field to establish the dependency relationships between different fields. Dependency inference focuses on the correlation among fields based on input file length, checksum, offset types, and other characteristics. FieldsFuzz determines the dependency relationships between these specialized fields and other data fields by extracting the taint instruction list of these fields and

Table 1
Semantic types of FieldFuzz defined.

Field type	Description
Magic number	Magic bytes are used to identify the beginning and end of fields or the file types, usually with fixed valid values.
Enumeration	The valid values for the enumerated field are strictly confined to a predetermined, fixed-size set of options.
Length	The length field represents the length of a data block composed of one or more fields in the input file. Usually used for reading fields and loop processing
Checksum	The checksum field is used to verify the integrity of some data in the input and needs to be compared with the results calculated from other fields.
Offset	The offset field represents the position of a data field in the file, used to read or manipulate a specific location of the field.
Data	Valid data content fields of the file

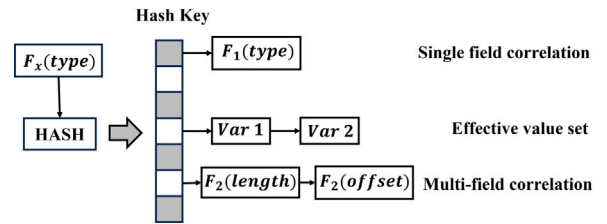


Fig. 5. Field dependency table storage mode.

tracking the byte fields affected by the operands of *cmp* and *load* instructions. For instance, the checksum field is compared with a variable, and the tainted byte field associated with this variable typically corresponds to the field verified by the checksum field.

To efficiently implement insertion and lookup operations for field relationships, FieldsFuzz constructs a field dependency table using a hash table to store these relationships. The structure of the field dependency table, as illustrated in Fig. 5, records the relevant fields and their corresponding valid value sets for different categories. For Magic number and Enumeration fields, the dependency table stores their valid value sets. The Length, Checksum, and Offset fields are recorded in a hash table to indicate their functional scope. When certain fields have multiple dependent fields or valid values, these values are stored in a linked list format. Based on the field dependency table, it is possible to quickly add and query inter-field dependencies.

The structure of the Field dependency table is illustrated in Fig. 5. To facilitate rapid insertion and lookup of dependency relationships, FieldsFuzz first extracts the initial position and length of the dependency relationship $F_x - F_y$ or the effective value relationship $F_x - Var_y$. It then employs a hash function to construct a hash key for the field. This generated hash key serves as an index for storing dependency or valid value relationships within the corresponding hash bucket. Each hash bucket maintains a linked list that records the valid dependency fields or valid values of F_x . When multiple dependent fields or valid values exist within a field, each dependent field or valid value is encapsulated as an independent node and linked to the list in the order of discovery. This storage method not only enables efficient insertion and query operations but also allows for the retrieval of all valid values or dependencies by traversing the linked list during the mutation process.

3.2. Format based mutation

Based on taint analysis, FieldsFuzz generates a file format tree and a dependency table, utilizing a file format mutation method to enhance the efficiency of fuzz testing. Building on the AFL mutation rules,

the file format mutation technique has been refined to incorporate dependency-based mutation and file structure-based mutation methods.

A simple mutation in an input field not only changes its own properties but also alters its parent field and dependent fields. If modifications made to fields during fuzz mutation disrupt the overall structure of the input, the generated input is likely to be rejected by the program, preventing exploration of the deeper code space. The dependency-based mutation method aims to achieve field-level mutation rather than being confined to byte-level mutation. During the field mutation process, the dependent fields of the mutated fields are modified according to specific rules to ensure the validity of the mutation testcases. Additionally, recognizing that certain deep vulnerabilities may be triggered by malformed files, FieldsFuzz employs a structural damage mutation method to create test cases to explore unknown vulnerabilities.

Algorithm 2 outlines the mutation method based on dependency relationships. The algorithm begins by employing a Depth-First Search method to traverse the file format tree, identifying the single minimum field of input. It then adds each leaf node of the file format tree to the mutation queue and queries the dependency relationship table to ascertain the dependency relationship between the node in the identified field and its parent node, subsequently adding this information to the field dependency relationship queue. Subsequently, mutation operations are conducted based on the rule set governing field dependency relationships. In this process, FieldsFuzz categorizes the mutation operation into two distinct types: exploration mutation and exploitation mutation. Exploration mutation preserves the dependency relationship during input mutation, meaning that the modification of fields simultaneously adjusts the dependency relationship between the field node and its parent node. In contrast, exploitation mutations can sever dependency relationships, alter input structures, and expose vulnerabilities when testing programs that handle abnormal inputs during the mutation process. Table 2 outlines the mutation rules defined by FieldsFuzz for various types of fields.

Algorithm 2 Mutation Method based on Fields Dependency

```

1: function MUTATION(root)
2:   Search_Stack ← new stack()
3:   Search_Stack.push(root)
4:   Path_Stack ← new stack()
5:   while Search_Stack is not empty do
6:     Cur_Node ← Search_Stack.pop()
7:     Cur_Depend ← Depend_Stack.pop()
8:     Cur_Depend.append(Find_Depend(Cur_node))
9:     if Cur_Node.Sub_list is empty then
10:      Field_Mutation(Cur_Node, Cur_Node.type,
11:        Cur_Depend)
12:     else
13:       for each Sub_Node in Cur_Node.Sub_List do
14:         Search_Stack.push(Sub_Node)
15:         Path_Stack.push(Cur_Depend)
16:       end for
17:     end if
18:   end while
19: end function

```

Based on the file structure mutation method, mutation is accomplished by altering the file format's tree structure. During this process, FieldsFuzz systematically disrupts the file structure by deleting, swapping, copying, and inserting nodes within the format tree in the data field, thereby achieving structural mutation. This approach to structural mutation can thoroughly investigate and reveal potential unknown vulnerabilities. The specific implementation rules are outlined below, and Fig. 6 illustrates these rules.

- **Delete Nodes:** Randomly select specific nodes and remove both the node and its corresponding child node fields from the file structure.

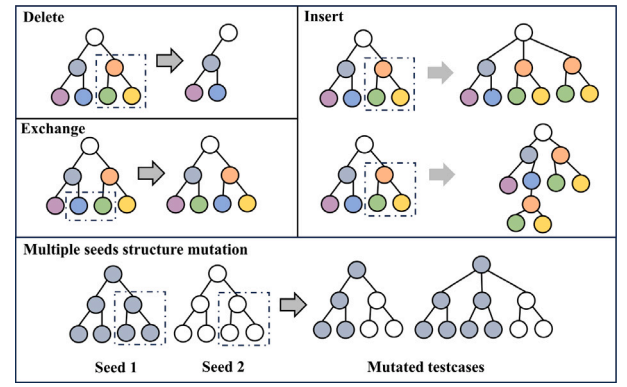


Fig. 6. Seed structure mutation rules.

- **Swap Nodes:** Exchange the positions of two nodes and modify the arrangement of fields in the input file.
- **Insert Node:** Copy the content of a node and randomly select a location to insert it, thereby introducing redundant data or irregular structures.

In addition to the structural mutation method for individual seed files, FieldsFuzz also establishes intelligent stitching seed structure mutation rules. This process is illustrated in Fig. 6. Given the seed file s_1 , select node n_1 from the s_1 file format tree. Next, choose a second seed file s_2 and select the corresponding node n_2 from the file format tree in s_2 , ensuring that n_1 and n_2 are of the same type. FieldsFuzz then copies the relevant field of node n_2 in s_2 to the end of the existing node n_1 in the seed file s_1 , or it may completely replace the content of the n_1 field to achieve the effect of cross-mutation. Simultaneously, adjusting field dependencies based on the dependency table implements either exploration or exploitation mutation.

4. Implementation and experiment

We have developed a dynamic taint analysis engine based on Pin 3.7 to implement instruction tracing. Additionally, we implemented taint instruction handling, as well as the construction of a file format tree and a dependency table, using approximately 2.8k lines of C++ code. The fuzz testing module is built on AFL 2.57b and incorporates mutation strategies based on fields and dependencies. Furthermore, we created a Python script to manage the scheduling between input structure recognition and input mutation.

To evaluate the effectiveness of FieldsFuzz, we designed and conducted comprehensive experiments to address the following research question:

- **RQ 1:** How accurately can FieldsFuzz identify file fields, and how efficient is it in doing so?
- **RQ 2:** How does FieldsFuzz compare to state-of-the-art fuzzers regarding path coverage and the efficiency of vulnerability discovery?
- **RQ 3:** How effective are the file format tree and dependency table proposed by FieldsFuzz?

Experimental Setup. We conducted our experiment on a server equipped with Intel Xeon® Silver 4214 CPUs operating at 2.20 GHz and 128 GB of RAM, running Ubuntu 22.04. For each program in the benchmark test, we randomly selected an initial corpus and utilized the same seed for all fuzzers. Under identical configurations, we executed the fuzzers on each target application, ensuring that each fuzzer was allocated to a single CPU core to minimize the impact on the performance of the test environment.

Table 2
Rules of field-based mutation implemented by FieldsFuzz.

Fields type	Exploration mutation	Exploitation mutation
Magic number	Keep the Magic number value fixed	–
Enumeration	The enumeration field changes in the set of valid values	–
Length	Randomly increase and decrease the length value, while randomly increasing or deleting the corresponding data field byte count, so that the length is equal to the data length	Only change length Set the value of Length to negative or maximum
Checksum	Keep the checksum and its corresponding data fields unchanged.	Checksum value randomly changes
Offset	Randomly increase and decrease the offset value, while randomly increasing or deleting the corresponding data field byte count, so that the length is equal to the offset length	Only change offset Set the value of Offset to negative or maximum
Data	Flip,delete,add data field bytes, and modify corresponding dependency relationships	Randomly flip, delete, and add bytes to this field

Table 3
The benchmark programs and parameters used in evaluation.

Subjects	Version	Format
nm -C @@	binutils-2.32	elf
objdump -d @@	binutils-2.32	elf
xmllint -testIO @@	libxml2 2.11.0	xml
pdftotext @@ /dev/null	xpdf 4.00	pdf
tcpdump -e -vv -nr @@	libpcap 1.8.1	pcap
bsdtar -xf @@ /dev/null	libarchive-3.4.0	tar
gif2png @@	gif2png 2.5.8	gif
readpng @@	libpng-1.6.37	png
jhead @@	jhead-3.03	jpeg
exiv2 @@ /dev/null	exiv2-0.26	jpeg
mp3gain @@	mp3gain-1.5.2	mp3
mp42aac @@ /dev/null	bento4-1.5.1-628	mp4

Benchmark Program. We mainly based our benchmark testing program on the fuzz testing benchmarks FuzzBench (Metzman et al., 2021) and UniFuzz (Li et al., 2021), along with other research papers on fuzz testing tools, to establish our testing objectives. The primary criteria for selecting programs were to include a diverse range of programs with various file formats as inputs. We chose 12 programs, detailed in Table 3, which have been meticulously selected as benchmarks to assess the effectiveness and efficiency of the fuzz testing method. These programs encompass a broad spectrum of functionalities, including different types of input formats such as image processing, audio parsing, video manipulation, compressed file handling, and binary file processing. Notably, these programs not only demonstrate functional variety but also possess multiple security vulnerabilities, including common issues like heap buffer overflow, stack overflow, segmentation faults, global buffer overflow, various forms of stack buffer overflow, memory leaks, use-after-free errors, and floating-point operation exceptions.

Baseline Fuzzer. In our comparative experiment, we selected five state-of-the-art open-source fuzzers as evaluation benchmarks. These include the generic fuzzers AFL (version 2.57b) and AFL++, as well as the input structure-aware fuzzers WEIZZ, ProFuzzer, and NestFuzz. AFL is a widely used universal fuzz testing tool. AFL++ integrated multiple fuzz testing methodologies to optimize seed scheduling and energy management strategies. It is one of the highest-rated fuzz testing tools according to FuzzBench. WEIZZ, NestFuzz, and ProFuzzer are all format-aware fuzz testing tools.

4.1. RQ1: Accuracy and effectivity of input fields recognition

To evaluate the performance of FieldsFuzz in identifying fields of input files, we compared it with other input structure inference

methods, including WEIZZ, ProFuzzer, and NestFuzz. Among these, WEIZZ and ProFuzzer employ a byte-by-byte flipping method, while both NestFuzz and FieldsFuzz utilize dynamic taint analysis to identify the relevant input fields.

In this experiment, we utilized four format-aware fuzz testing tools to evaluate the twelve programs mentioned above, primarily assessing the tools' ability to recognize input file boundaries and categories. For each testing program, we selected ten initial seeds of varying sizes and categorized them into small and large seeds based on their dimensions. We recorded the average analysis speed and average accuracy of the fuzzing testing tool for the input file, respectively.

To accurately calculate the effectiveness of field recognition, we first preprocess the input files. For each file, we utilize a standardized format template from 010 Editor to parse the file and export all field records, including boundary and type information. During this process, we manually mark any incorrectly identified fields in the template and remove redundant fields. Next, we designate the field type based on the function of field division among various fuzz testing tools, we define the boundary accuracy as the number of correctly recognized fields by the fuzz testing tool divided by the total number of recognized fields by the tool. For field categories, we compare the fuzz testing tool defined field types with the actual functions of the field to determine if they are correct. Type accuracy is defined as the number of correctly identified field types divided by the total number of recognized fields by the tool. Due to WEIZZ only recognizing checksum fields, the accuracy of WEIZZ's field type recognition is not included in the statistics. The accuracy of identifying input file fields through fuzz testing attacks is shown in Table 4. The results showed that FieldsFuzz had an average accuracy of 96.23% and 94.13% in field recognition and type recognition for small-sized seeds, and 95.89% and 94.93% for large-sized seeds, respectively, which were higher than other comparative fuzzers.

The Fig. 7 illustrates the effectiveness of various fuzz testing tools in identifying the header field of GIF files. The white field, represented by the bytes 0x20, 0x00, 0x20, and 0x00 in the first line of the GIF file, indicates the length and width of the GIF image. WEIZZ treats these two length fields as a single field during field recognition, disregarding their distinct roles. The yellow field represents the global color table of the GIF file, where every three bytes correspond to a specific color. There is a partial misclassification in the WEIZZ recognition process. NestFuzz accurately identified most of the fields; however, it overlooked the fields 0x21, 0xF9, and 0x2C, as depicted in Fig. 7(b). The byte identification of these fields signifies their starting positions, suggesting that NestFuzz has limitations in field recognition and may miss some magic byte fields identifications. In contrast, FieldsFuzz not only demonstrates high accuracy in recognizing file fields but also yields comprehensive recognition results.

Table 4

Field boundary accuracy/type accuracy comparison among the different input format-based fuzzers.

Program	Format	Small seeds					Large seeds				
		Size(byte) ^a	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz	Size(byte)	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz
nm	elf	324	39.40%/68.25%	94.30%/(N/A) ^b	98.16%/87.42%	99.08%/92.36%	1806	63.85%/66.67%	92.40%/(N/A)	97.90%/89.93%	97.50%/96.80%
objdump	elf	324	57.17%/78.85%	95.25%/(N/A)	95.32%/93.61%	95.36%/93.24%	1806	61.85%/76.93%	93.31%/(N/A)	96.24%/94.19%	98.50%/97.10%
xmllint	xml	75	60.61%/77.28%	87.32%/(N/A)	84.32%/82.85%	90.80%/90.80%	823	67.15%/75.38%	87.54%/(N/A)	86.75%/85.10%	92.91%/91.29%
pdftotext	pdf	987	66.32%/86.87%	91.23%/(N/A)	85.49%/94.65%	96.72%/91.33%	8641	80.71%/80.71%	95.23%/(N/A)	88.93%/93.08%	92.96%/98.84%
tcpdump	pcap	894	37.71%/73.54%	89.41%/(N/A)	94.81%/91.39%	98.51%/94.67%	6740	61.81%/71.53%	82.92%/(N/A)	94.29%/93.62%	99.06%/95.12%
bsdtar	tar	846	59.21%/62.26%	83.12%/(N/A)	88.82%/91.59%	92.00%/92.61%	5617	60.82%/69.84%	91.45%/(N/A)	96.25%/92.04%	97.97%/90.12%
gif2png	gif	198	81.79%/71.86%	79.56%/(N/A)	89.34%/94.73%	96.71%/95.57%	6612	81.35%/72.17%	87.97%/(N/A)	96.88%/94.05%	97.52%/98.92%
readpng	png	1079	74.59%/67.38%	94.37%/(N/A)	92.86%/97.43%	100%/93.21%	4897	83.49%/64.39%	92.47%/(N/A)	94.32%/95.17%	97.65%/92.23%
jhead	jpeg	413	58.77%/72.89%	84.75%/(N/A)	92.10%/92.66%	97.55%/98.64%	3212	82.22%/66.08%	85.46%/(N/A)	91.34%/94.08%	94.86%/96.72%
exiv2	jpeg	413	63.65%/69.67%	82.39%/(N/A)	92.39%/97.44%	97.55%/98.64%	3212	72.66%/68.27%	86.59%/(N/A)	92.83%/93.30%	94.86%/95.47%
mp3gain	mp3	1221	68.32%/89.58%	84.50%/(N/A)	90.29%/98.61%	97.87%/94.54%	6857	61.24%/87.56%	83.50%/(N/A)	88.32%/89.28%	95.57%/93.67%
mp4aac	mp4	1267	77.25%/46.23%	86.41%/(N/A)	96.95%/91.90%	92.60%/91.90%	9701	79.34%/48.76%	85.72%/(N/A)	97.64%/94.47%	91.30%/92.92%
Average			62.07%/72.06%	87.72%/(N/A)	91.74%/92.86%	96.23%/94.13%		71.37%/70.69%	88.71%/(N/A)	93.47%/92.36%	95.89%/94.93%

^a The Size in the table is the average of 5 different seed sizes.^b WEIZZ cannot recognize field types completely.**Table 5**

Average time to process an input (seconds).

Program	Format	Small input					Large input				
		Size(byte)	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz	Size(byte)	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz
nm	elf	324	873.24	31.63	0.19	0.12	1806	1321.12	49.94	0.20	0.14
objdump	elf	324	865.21	38.13	0.23	0.12	1806	1354.74	53.29	0.23	0.14
xmllint	xml	75	439.60	17.33	0.19	0.10	823	974.16	26.48	0.19	0.16
pdftotext	pdf	987	519.55	46.98	2.85	1.78	8641	1432.21	76.16	2.91	1.76
tcpdump	pcap	894	574.66	48.94	0.65	0.66	6740	2675.15	65.20	0.70	0.67
bsdtar	tar	846	625.09	32.14	0.06	0.06	5617	1837.21	58.61	0.06	0.06
gif2png	gif	198	388.82	50.07	0.07	0.08	6612	1749.56	83.19	0.08	0.08
readpng	png	1079	1436.54	70.58	0.23	0.13	4897	4867.58	87.21	0.27	0.19
jhead	jpeg	413	236.87	45.36	0.15	0.10	3212	1876.54	154.79	0.20	0.12
exiv2	jpeg	413	271.57	42.93	0.17	0.12	3212	1901.45	151.33	0.19	0.18
mp3gain	mp3	1221	2986.87	143.75	0.89	1.02	6857	10 234.28	258.95	0.95	1.16
mp4aac	mp4	1267	3002.83	236.78	1.57	1.05	9701	16 408.56	414.66	1.63	1.18
Average			1018.40	67.05	0.60	0.45		3886.05	123.32	0.63	0.49

[47 49 46]	[38 39 61]	[20 00 20 00]	[F2 01 00]	[66 CC CC]
FF FF FF	[00 00 00]	[33 99 66 99]	FF CC	[00 00 00]
00 00	[00 00 00]	[21 F9]	[04 00 00 00 00 00]	[2C] [00 00]
[00 00]	[20 00 20 00]			

(a)WEIZZ

[47 49 46]	[38 39 61]	[20 00]	[20 00]	[F2 01 00]	[66 CC CC]
FF FF FF	00 00 00	33 99 66 99	FF CC	00 00 00 00	00 00 00
00 00 00 00 00	[21 F9]	[04 00 00 00 00 00]	2C	[00 00]	
[00 00]	[20 00]	[20 00]			

(b)NestFuzz

[47 49 46]	[38 39 61]	[20 00]	[20 00]	[F2 01 00]	[66 CC CC]
FF FF FF	00 00 00	33 99 66 99	FF CC	00 00 00 00	00 00 00
00 00 00 00 00	[21 F9]	[04 00 00 00 00 00]	[2C]	[00 00]	
[00 00]	[20 00]	[20 00]			

(c)FieldsFuzz

Fig. 7. Identification of GIF file header results using different fuzzers. Bytes in red brackets indicate they are different than the specification, while the shallow parts indicate the target fails to extract the fields. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

In order to evaluate the speed of FieldsFuzz in recognizing file formats, we recorded the time required for the fuzzy testing tool to identify the input file format during the experiment, as shown in Table 5. The term “process an input” refers to the total duration from the initiation of processing the input file by the fuzz testing tool to the complete output of the initial seed format record. From the results in Table 5, we can draw the following conclusions: Among all the programs, FieldsFuzz has the shortest average time for recognizing the input file field, with average times of 0.60 s and 0.63 s for testing small-sized and large-sized seeds, respectively. The byte-by-byte flipping method

employed by ProFuzzer and WEIZZ to identify file regions results in significantly higher recognition times compared to FieldsFuzz, with increased time consumption as the initial seed size grows. In contrast, NestFuzz and FieldsFuzz utilize taint analysis techniques that are not sensitive to input file size. Compared to NestFuzz, FieldsFuzz is more efficient in identifying fields, with average reductions in testing times of 0.15 s and 0.14 s for small-sized and large-sized seeds, respectively. The recognition efficiency in programs such as pdftotext has been significantly enhanced due to FieldsFuzz’s streamlined merging and rule matching of stain records.

4.2. RQ2: Edge coverage

In this experiment, we conducted a 24-hour fuzz test using a baseline fuzzer as part of the benchmark testing program. We selected different initial seed sets for five repeated experiments to minimize the impact of randomness. The experiment recorded these program’s edge coverage averaged over the 24-hour period. The results are presented in Table 6.

As shown in Table 6, FieldsFuzz achieved the highest edge coverage among 7 out of 12 benchmark test programs. Specifically, compared to AFL, AFL+, ProFuzzer, WEIZZ, and NestFuzz, FieldsFuzz improved edge coverage by 30.30%, 11.88%, 21.53%, 15.50%, and 5.95%, respectively. Notably, format-aware fuzz testing tools have not performed well in xmllint, particularly because XML file reading and parsing prioritize XML syntax over file fields.

To further demonstrate the effectiveness of FieldsFuzz’s edge exploration, we tracked the growth trend of edge coverage of the fuzz testing tool throughout the testing process, as illustrated in Fig. 8. In most projects, FieldsFuzz achieves the highest edge coverage at a faster rate compared to all other fuzzers. On one hand, FieldsFuzz exhibits a quicker parsing speed for initial input file fields than other fuzzing tools that rely on file structure awareness. On the other hand, FieldsFuzz generates more effective seeds through a file format mutation strategy,

Table 6
Number of edge covered by each fuzzer on real-world programs.

Program	AFL	AFL++	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz
nm	5017	7557	7433	7321	7668	8145
objdump	6924	7062	6258	6495	7590	7765
xmllint	6601	7812	6945	7356	7424	7623
pdftotext	1220	1658	1707	1920	1923	2208
tcpdump	12 284	13 872	11 294	13 109	13 421	14 567
bsdtar	6147	7021	6594	6689	7486	7401
gif2png	3201	3479	3454	3501	4010	4805
readpng	1931	2075	2210	2218	1946	2122
jhead	535	598	547	565	584	587
exiv2	3978	4761	4891	4863	6552	6947
mp3gain	1204	1324	1178	1242	1307	1578
mp4aac	2804	3166	3079	3210	3854	3808
Total	51846	60385	55590	58489	63765	67556
Decrease	(−30.30%)	(−11.88%)	(−21.53%)	(−15.50%)	(−5.95%)	

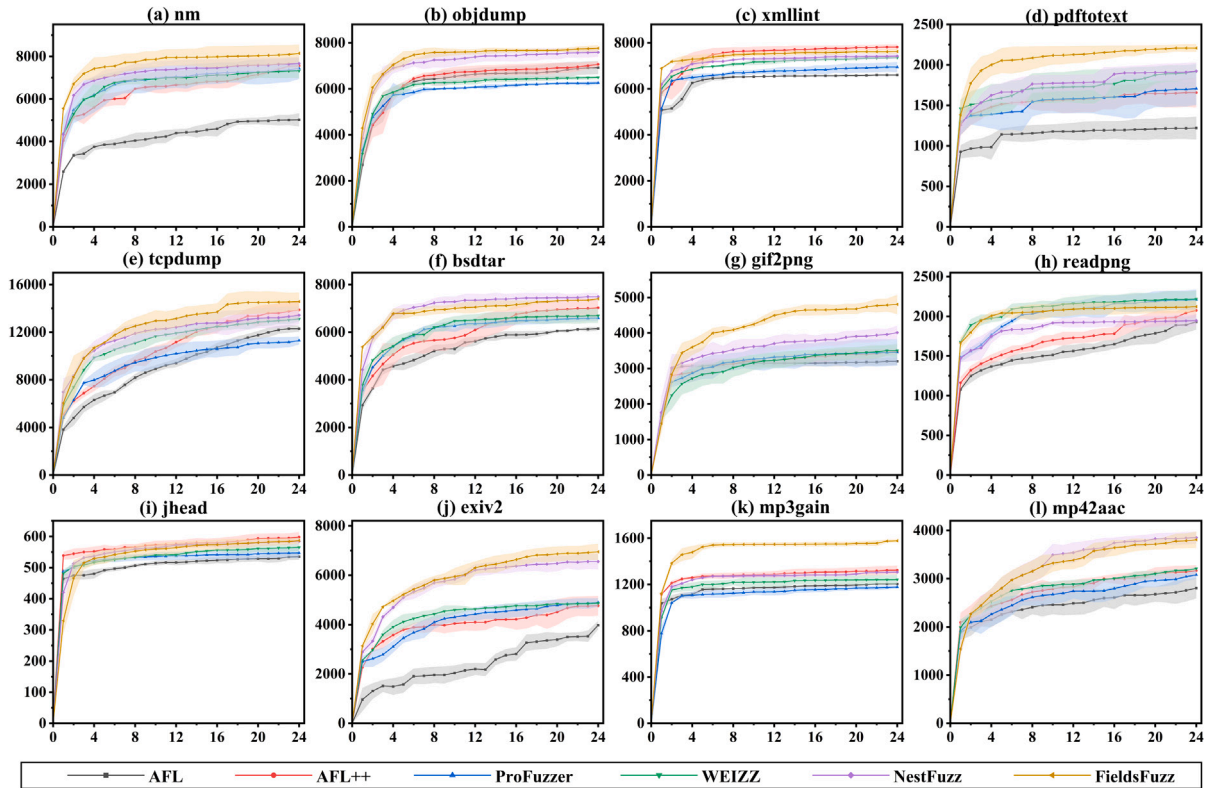


Fig. 8. The arithmetic mean edge coverage of each fuzzer and one standard deviation error bars over five runs.

in contrast to testing tools such as AFL, thereby reducing the number of invalid mutations. Consequently, in terms of coverage changes, FieldsFuzz enhances edge coverage more rapidly.

4.3. RQ2: Vulnerability detection

In order to evaluate the vulnerability detection capabilities of the fuzz testing tool, we conducted a 24-hour fuzz test using a baseline fuzzer within the benchmark testing program. We enabled AddressSanitizer (ASAN) during program compilation to identify and eliminate duplicate vulnerability discovery inputs from the testing tool. Additionally, we recorded the unique number of vulnerabilities identified by each testing tool in the program.

Table 7 presents the number of unique errors identified during five runs of various fuzz testing tools. FieldsFuzz has detected the highest number of unique errors across most programs. Specifically,

FieldsFuzz uncovered 415 unique bugs, resulting in an increase in vulnerability discoveries of 139.88%, 27.30%, 46.64%, 28.48%, and 10.96% compared to AFL, AFL+, ProFuzzer, WEIZZ, and NestFuzz, respectively.

To better illustrate FieldsFuzz's capability in identifying vulnerabilities, we compared the unique vulnerabilities discovered by the fuzz testing tool with those listed in the vulnerability library. Table 8 presents the top three programs with the highest number of CVE vulnerabilities identified by the fuzz testing tool, along with their vulnerability discovery status. The results indicate that FieldsFuzz outperforms other fuzzers in identifying CVEs, successfully uncovering a total of 21 CVE IDs. It is three more than the second-ranked tool, NestFuzz. These CVE IDs include denial-of-service and stack overflow vulnerabilities, which pose significant threats to businesses and users. In comparison to the original AFL tool, FieldsFuzz identified an additional 10 CVE IDs across all test targets, demonstrating the

Table 7
Number of unique bugs achieved by various fuzzers.

Program	AFL	AFL++	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz
nm	7	10	13	12	12	15
objdump	14	23	20	24	22	28
xmllint	0	2	0	0	0	1
pdftotext	0	1	1	0	1	1
tcpdump	44	87	79	98	159	162
bsdtar	0	0	0	0	2	1
gif2png	26	78	70	70	72	78
readpng	3	6	5	7	3	7
jhead	5	18	10	16	16	12
exiv2	7	11	10	10	11	15
mp3gain	50	67	53	62	48	69
mp42aac	17	23	22	24	28	26
Total	173	326	283	323	374	415
Decrease	(−139.88%)	(−27.30%)	(−46.64%)	(−28.48%)	(−10.96%)	

Table 8
The published CVE IDs found by each fuzzer.

CVE ID	AFL	AFL++	ProFuzzer	WEIZZ	NestFuzz	FieldsFuzz
tcpdump	CVE-2016-7973	•	•	•	•	•
	CVE-2016-7985	•	•	•	•	•
	CVE-2016-7983	•	•	•	•	•
	CVE-2017-13028	•	•			•
	CVE-2017-13687		•	•	•	•
	CVE-2017-13688		•	•	•	•
	CVE-2017-13689			•	•	•
	CVE-2017-13690		•		•	•
	CVE-2018-14462		•	•		•
mp3gain	CVE-2017-14406	•	•	•	•	•
	CVE-2017-14407	•	•	•	•	•
	CVE-2018-10777	•	•			•
	CVE-2018-10783		•	•	•	•
	CVE-2018-10784		•	•	•	•
mp42aac	CVE-2018-10791	•	•	•	•	•
	CVE-2018-10790	•	•	•	•	•
	CVE-2018-10785		•	•	•	•
	CVE-2018-14587	•	•	•	•	•
	CVE-2018-14584	•	•	•	•	•
	CVE-2018-14531			•	•	•
	CVE-2018-14588		•	•	•	•
total	11	16	13	14	18	21

effectiveness of our approach in enhancing vulnerability detection efficiency. The impressive performance in discovering CVE vulnerabilities underscores the effectiveness and efficiency of FieldsFuzz in detecting critical vulnerabilities.

4.4. RQ3: Validity of format tree and dependency table

The file format tree constructed by FieldsFuzz serves as the foundation for the mutation strategy. Accurately restoring the underlying structure of files is essential for keeping data fields during mutation and refactoring test cases. In this section, we use JPG files as examples to illustrate the correctness of the syntax structure recovered through our method.

We utilize FieldsFuzz to test *jhead*, analyze the JPG file format, and generate a visual representation of the file format tree. We document the starting and ending bytes of the file format tree nodes as labels. Fig. 9 illustrates the format tree for the seed file *not_kitty.jpg*, specifically the APP0 field and the DQT dat[0] field. Additionally, we employed 010 Editor to parse the seed file *not_kitty.jpg* provided by AFL and recorded its field divisions, as shown in Table 9. By comparing and analyzing the generated results, we can confirm that FieldsFuzz largely reconstructs the syntax structure corresponding to the test cases.

Although the tool demonstrates high accuracy and consistency, there are some discrepancies in recognition in specific fields due to the program not fully adhering to the specifications for the file parsing process. FieldsFuzz employs a stain analysis method to identify the program's processing steps, and the recognized file structure aligns with the program's parsing methodology. We exported the dependency table generated by FieldsFuzz as a JSON file. Fig. 10 presents some of the Magic Number, Enumeration, and Length field dependency records. According to the results presented in Fig. 10, FieldsFuzz effectively records the valid values or dependent field for three types of fields. This indicates that the field dependency table can accurately identify field types and document dependency records.

In addition to JPG files, we conducted a comprehensive analysis of PNG, GIF, MP3, MP4, and ELF file formats during the experiment. The experiment selects five seeds for each format program as initial inputs for fuzz testing and records the generated file format tree and dependency table information separately. Simultaneously, we utilized 010 Editor to manually analyze the input file structure and dependency relationships, comparing them with the file format tree and region dependency table generated by FieldsFuzz. This comparison aims to verify FieldsFuzz's capability to partition file formats and recognize dependencies. To assess the effectiveness of the method proposed in

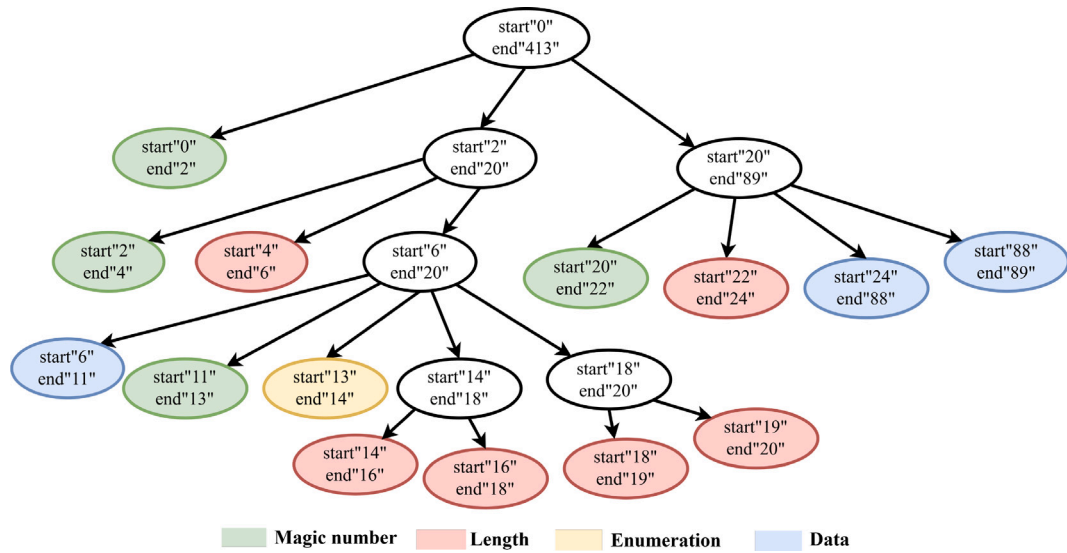


Fig. 9. The JPG header format tree parsed by FieldsFuzz.

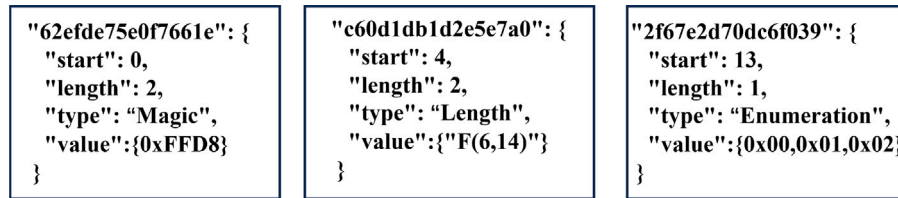


Fig. 10. Partial field dependency records parsed by FieldsFuzz.

Table 9
JPG file header format standard.

	Field name	Start byte	End byte	Type
APP0 app0	enum M_ID SOIMarker	0	2	Magic number
	enum M_ID marker	2	2	Magic number
	WORD szSection	4	6	Length
	char AppoType[5]	6	11	Data
	short versionHigh	11	13	Magic number
	short versionLow	11	13	Magic number
	ubyte units	13	14	Enumeration
	WORD Xdensity	14	16	Length
	WORD Ydensity	16	18	Length
	ubyte xThumbnail	18	19	Length
	ubyte yThumbnail	19	20	Length
DQT dat[0]	enum M_ID marker	20	22	Magic number
	WORD szSection	22	24	Length
	struct QuanTable qtable	24	89	Data

this paper, commonly used comprehensive evaluation indicators were selected to evaluate both the effectiveness and the completeness of recognition of the format tree and dependency table. The parsing results for field division in the file format tree are presented in Table 10, which illustrates the field recognition accuracy and the number of error nodes across a total of five experiments. The data in Table 10 indicate that FieldsFuzz achieves an average node recognition accuracy of 92.12% across six programs, with each program attaining an accuracy exceeding 90%. Further analysis of the error nodes encountered during the identification process revealed that these errors primarily consist of unrecognized fields, incorrect boundary divisions of fields, and instances where subfields were not delineated. These issues largely arise from certain subfields that are not utilized during program execution or from errors generated during instruction parsing. Nevertheless, the overall accuracy of FieldsFuzz's field partitioning remains high.

To evaluate the effectiveness of dependency recognition in the dependency table, we manually analyzed field types and dependency relationships for input seeds based on input format templates and file specifications. This analysis included relationships such as Magic Byte, Enumeration, Offset, Length, and Checksum. We compared these with the relationships identified in the FieldsFuzz dependency table and documented the results of the dependency recognition process. True Positional Dependencies indicate that a file possesses a specific dependency relationship that has been correctly recognized by FieldsFuzz. In contrast, False Negative Dependencies signify that a file has a dependency relationship that was not recognized by FieldsFuzz. During our analysis of the dependency relationship table results, we discovered that, in addition to the region dependencies present in 010 Editor, certain path judgment conditions generated during program execution were also recorded. For instance, in the *gif2png* program,

Table 10

The result of FieldsFuzz file format tree parsing.

File format	Program	Number of leaf nodes in all format trees	Wrong number of leaf nodes	Accuracy	Fields range division error	Unclassified subfields	Unrecognized fields
jpg	jhead	663	56	91.55%	23	25	8
png	readpng	578	49	91.52%	6	17	26
gif	gif2png	248	18	92.74%	7	7	4
mp3	mp3gain	2243	154	93.13%	46	77	31
mp4	mp42aac	3121	216	93.08%	72	87	57
elf	nm	547	51	90.68%	18	12	21
total		7400	544	92.12%	172	225	147

Table 11

The results of FieldsFuzz dependency recognition.

File format	Program	Number of dependencies in 010 editor	Number of dependencies identified by FieldsFuzz	Number of true-positive dependencies	Number of false-negative dependencies	Number of extra dependencies	Number of valid dependencies	Number of wrong dependencies
jpg	jhead	432	492	379	53	22	401	91
png	readpng	417	467	362	55	18	380	87
gif	gif2png	174	193	132	42	27	159	34
mp3	mp3gain	1476	1675	1271	205	92	1363	312
mp4	mp42aac	1893	1703	1557	336	87	1644	59
elf	nm	406	394	363	43	16	379	15
total		4798	4924	4064	734	262	4326	598

there is a variable named *gray* that is calculated from the global RGB color table of the GIF file. The program utilizes the value of *gray* to determine whether the GIF file is a gray scale image, executing different logical branches based on this determination. Although this dependency relationship is not explicitly outlined in the GIF file specification, FieldsFuzz successfully identified the RGB color table fields during input testing and classified them as Checksum fields, while also recording their valid values. This finding suggests that the dependency condition is valid; therefore, it is marked as an additional dependency when documenting the results.

Table 11 presents the results of FieldsFuzz's dependency recognition. The test results indicate that FieldsFuzz successfully identified 4,064 format dependency specifications across the six target programs, which represents approximately 84.70% of the total file format definition specifications. Conversely, the remaining 15.30% of format dependencies were not detected. Additionally, FieldsFuzz identified 262 path-related dependency conditions, accounting for 5.32% of the overall recognition results. In summary, the output results of FieldsFuzz revealed that the proportion of effective dependencies reached 87.86%, while the proportion of erroneous dependencies, attributed to regional partitioning errors or recognition biases, was 12.14%.

The experimental results indicate that the dependency table generated by FieldsFuzz is effective in identifying dependency relationships within file formats and demonstrates high accuracy in dependency detection. Although FieldsFuzz is constrained by program functionality, seed execution paths, and other factors, it does produce some false positives and false negatives that may not fully align with the file format specifications. Nevertheless, in critical fields such as file headers and block lengths, the accuracy and comprehensiveness of the detection results are relatively high. Considering the experimental findings presented in Sections 4.2 and 4.3, these erroneous dependencies did not significantly impact the overall performance of FieldsFuzz. By further optimizing the algorithm and enhancing the region partitioning strategy, it is possible to reduce such errors and improve the recognition accuracy.

Based on our experimental results, we conclude that FieldsFuzz can effectively identify file formats and partial file region dependencies using file format trees and dependency tables.

5. Relatedwork

5.1. Taint-based fuzz

Dynamic taint analysis technology can identify the connection between input data and program state, as well as resolve intricate constraints within the program by tracing the data flow from its origin to its application. Numerous fuzz testing tools leverage taint analysis techniques to optimize input file scheduling and seed mutation, thereby improving fuzz testing efficiency. VUzzer (Rawat et al., 2017) employs application-aware fuzz testing technology to determine the comparison position and content at the *cmp* instruction through taint tracking and static analysis, effectively tackling multi-byte constraints and magic bytes during seed mutation. However, this approach requires a significant number of effective initial inputs. GREYONE (Gan et al., 2020) introduced Fuzzing driven Taint Inference (FTI), which alters the priority of byte mutations by mutating the input byte by byte and observing variables to establish the relationship between branches and input bytes. PATA (Liang et al., 2022) focuses on complex program constraints that are challenging to reach in fuzz testing, utilizing taint analysis to track changes in path execution variables, pinpoint key bytes, and execute specific mutations on those key bytes. Angora employs scalable byte-level taint tracking to address path constraints, recording the relationship between key bytes and paths, and using gradient descent methods to mutate key bytes for rapid path constraint resolution. Intriguer (Cho et al., 2019) minimizes resource consumption associated with solving symbolic execution constraints by using instruction-level taint analysis to document field behavior and create a field transformation tree to enhance constraint solving efficiency. Redqueen (Aschermann et al., 2019) monitors key instructions during execution and enhances fuzz testing through input state correspondence methods. By inserting hooks during program analysis, Redqueen observes *cmp* instructions and arithmetic operations to detect magic bytes and checksums. In contrast to these methods, FieldsFuzz utilizes data obtained from taint analysis for partitioning input file fields and recognizing types, thereby refining the seed mutation strategy in fuzz testing.

5.2. Format-aware fuzz

The format-aware fuzzer comprehends the input format during the fuzz testing process to enhance fuzzing efficiency. Identifying the boundaries of different fields within the input is fundamental to input format recognition. Some approaches attempt to utilize byte-by-byte mutation methods to segment input seeds into distinct fields. GRI-MOIRE (Blazytko et al., 2019) employs code coverage analysis to infer the format boundaries of the target program's input. It modifies certain bytes in the input and checks whether these alterations result in different code coverage. If the code coverage remains unchanged, the positions of the byte mutations can be randomly altered. Conversely, if there is a change in code coverage, the byte mutation probability according to the coverage change. ProFuzzer first defined six data types that cover the majority of input content. Then, flip the input file byte by byte, infer the type of each byte based on the coverage change characteristics generated by byte flipping, and merge consecutive bytes belonging to the same type. WEIZZ (Fioraldi et al., 2020a) identifies input file blocks through random byte mutation. During the byte flipping process of the input file in WEIZZ, it is determined which bytes have dependencies on a certain comparison instruction, and then the mutated portion of bytes is directed during the mutation process to identify the input file block based on the program state. Due to the non deterministic identification of individual fields or blocks during mutation execution, the recovery of file structure relies on deterministic rules, resulting in fragmented features of partitioned fields and ignoring field dependencies.

Another thread of format-aware fuzz testing tools typically generate structured inputs based on predefined templates. The Peach Fuzzer (Peachtech, 2021) utilizes XML format to define data models and state machines, automatically generating test cases that conform to specific formats based on these definitions. Sully (OpenRCE, 2012) creates complex protocol and data structure test sample templates for network protocol fuzz testing by defining blocks, requests, and other components. AFLSmart (Pham et al., 2019) employs format templates to parse input files, extract input structure fields and tree structures, and implement intelligent structure mutation strategies. FormatFuzzer (Dutra et al., 2023) generates test files using existing binary file templates. This method compiles templates into C++ code as input units for parsers, mutators, and efficient input generators, producing input files that adhere to formatting rules or mutate existing input files. However, generating structured input based on predefined templates often overlooks program parsing states, making it challenging to directed mutation in program file processing and lacking the capability to recognize unknown format files.

Based on dynamic taint analysis technology, several fuzz testing tools can infer input formats by recording the execution characteristics of inputs (Jain et al., 2018; Shi et al., 2023; Zhao et al., 2024; Deng et al., 2023). AIORE (Shi et al., 2023) employs taint analysis to identify the basic blocks responsible for processing each input byte and utilizes a minimum clustering algorithm to group these bytes. Additionally, AIORE leverages neural networks to infer the types of input fields based on the behavior of the basic blocks, thereby enhancing fuzz testing power scheduling based on input knowledge field types. ScissorFuzz (Zhao et al., 2024) uses lightweight dynamic taint tracking to identify the input field and streamline the size of test cases. In order to reduce the consumption of taint analysis, ScissorFuzz only tracks data movement instructions and abandons data conversion instructions during the taint analysis process. ScissorFuzz identifies the input field and structure by simplifying the taint tracking instruction, and then reduces the size of the input file based on the format tree. Although this process restores the field structure of the input file, it cannot recognize the field type and constraints. NestFuzz (Deng et al., 2023) uses DFSan to track taint information, models input processing logic, identifies length, type, and offset dependencies, and constructs an input processing tree to recognize structural dependencies. It adopts

a dependency based mutation method to improve testing efficiency. Although NestFuzz constructs an input processing tree based on program execution logic, the complexity of program instructions can lead to the presence of duplicate nodes within the generated tree. The input processing tree tends to be deep and its nodes are sparse, which diminishes the efficiency of storing and retrieving essential input structure and dependency information.

Unlike existing methods, FieldsFuzz constructs a file format tree, segments the input field, and identifies the boundaries and nested relationships within the input file. Simultaneously, FieldsFuzz employs a rule-based approach to process taint records, identify field types and inter-field dependencies, and create dependency tables. It utilizes a format-aware mutation method to minimize the number of invalid inputs and enhance the efficiency of fuzz testing.

6. Limitation and discussion

Although FieldsFuzz demonstrates good accuracy in identifying file fields and dependencies, it also has certain limitations. Firstly, FieldsFuzz employs taint analysis to track instructions, which can result in issues of insufficient or excessive contamination. For instance, some identifiers in the GIF file header are at the bit level, and due to FieldsFuzz's byte-level taint tracking, accurately identifying bit-level fields becomes challenging. Additionally, utilizing taint analysis in large-scale software testing may consume a significant amount of time and generate numerous redundant taint instruction records. The processing and streamlining of these taint records can adversely affect both testing efficiency and accuracy. The timing and frequency of taint analysis can significantly impact the efficiency of testing. Current strategies typically conduct taint analysis on each input seed during the initial execution to gather formatting information. However, the frequency of taint analysis execution may influence the accuracy of format tree construction. Therefore, optimizing the scheduling between taint analysis, format parsing, and fuzz testing is an area that requires improvement in the future.

Secondly, the file format tree can accurately represent file formats in most program tests, but when there are no nested or dependency relationships between input files, the file format tree may degenerate into a linear table, reducing traversal and mutation efficiency. FieldsFuzz can reflect the dependency relationships between fields when constructing a dependency table, but for complex constraints in the program, the dependency table cannot record all dependencies and complex constraints. Therefore, it is still necessary to solve complex constraints through symbolic execution methods (Jiang et al., 2023; Geretto et al., 2023). Due to variations in file format specifications and the processing capabilities of different programs, file format trees and dependency tables generally represent the specific format and constraints of the program used for parsing these files. These tools assist in developing fuzz testing variation strategies, which may not align with actual file specifications. If a precise analysis of the file format is required, it remains essential to conduct the analysis based on the pre-input dictionary file.

Finally, the structure aware mutation strategy improves the ability to generate effective seeds to a certain extent, but triggering many vulnerabilities requires input examples of structural destruction. In future work, we will design mutation scheduling and power scheduling strategies to optimize exploration and exploitation mutations and improve vulnerability detection efficiency (Lyu et al., 2019; Zhang et al., 2024; Jauernig et al., 2022; Su et al., 2024).

7. Conclusion

In this paper, we propose an enhanced format-aware fuzz testing tool, FieldsFuzz, designed to address the challenges posed by existing format-aware fuzz testing tools, which often struggle to recognize the complex structures and dependencies of seed files. This limitation

results in low efficiency when generating effective testcases. FieldsFuzz utilizes dynamic taint analysis to obtain execution information from seed files, analyzes and identifies field boundaries and field dependencies based on a predefined rule set, and constructs file format trees and field dependency tables. This approach allows for a more accurate representation of the structure of input files. FieldsFuzz mutates seed files based on file format tree, enabling the generation of more valid testcases and significantly improving the efficiency of fuzz testing. Our evaluation results demonstrate that FieldsFuzz outperforms other format-aware fuzz testing tools in terms of efficiency and accuracy in format recognition. Additionally, FieldsFuzz surpasses other advanced fuzz testing methods regarding code coverage and vulnerability discovery.

CRedit authorship contribution statement

Yinghao Su: Writing – review & editing, Software, Resources, Writing – original draft, Methodology. **Dapeng Xiong:** Funding acquisition, Visualization, Formal analysis, Project administration, Data curation. **Kechang Qian:** Data curation, Investigation, Formal analysis. **Yu Wang:** Validation, Investigation, Methodology, Supervision. **Qingyao Zeng:** Data curation, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Aschermann, C., Schumilo, S., Blazytko, T., Gawlik, R., Holz, T., 2019. REDQUEEN: Fuzzing with input-to-state correspondence. In: NDSS, vol. 19, pp. 1–15.
- Beaman, C., Redbourne, M., Mummery, J.D., Hakak, S., 2022. Fuzzing vulnerability discovery techniques: Survey, challenges and future directions. *Comput. Secur.* 120, 102813.
- Blazytko, T., Bishop, M., Aschermann, C., Cappos, J., Schlögel, M., Korshun, N., Abbasi, A., Schweighauser, M., Schinzel, S., Schumilo, S., et al., 2019. GRIMOIR: Synthesizing structure while fuzzing. In: 28th USENIX Security Symposium. USENIX Security 19, pp. 1985–2002.
- Cho, M., Kim, S., Kwon, T., 2019. Intriguer: Field-level constraint solving for hybrid fuzzing. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 515–530.
- Cui, B., Wang, F., Hao, Y., Wang, L., 2016. A taint based approach for automatic reverse engineering of gray-box file formats. *Soft Comput.* 20, 3563–3578.
- Deng, P., Yang, Z., Zhang, L., Yang, G., Hong, W., Zhang, Y., Yang, M., 2023. NestFuzz: Enhancing fuzzing with comprehensive understanding of input processing logic. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 1272–1286.
- Dutra, R., Gopinath, R., Zeller, A., 2023. Formatfuzzer: Effective fuzzing of binary file formats. *ACM Trans. Softw. Eng. Methodol.* 33 (2), 1–29.
- Fioraldi, A., D’Elia, D.C., Coppa, E., 2020a. WEIZZ: Automatic grey-box fuzzing for structured binary formats. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1–13.
- Fioraldi, A., Maier, D., Eißfeldt, H., Heuse, M., 2020b. AFL++: Combining incremental steps of fuzzing research. In: 14th USENIX Workshop on Offensive Technologies. WOOT 20.
- Gan, S., Zhang, C., Chen, P., Zhao, B., Qin, X., Wu, D., Chen, Z., 2020. GREYONE: Data flow sensitive fuzzing. In: 29th USENIX Security Symposium. USENIX Security 20, pp. 2577–2594.
- Geretto, E., Hohnerlein, J., Giuffrida, C., Bos, H., Van Der Kouwe, E., v. Gleissenthall, K., 2023. Triereme: Speeding up hybrid fuzzing through efficient query scheduling. In: Proceedings of the 39th Annual Computer Security Applications Conference. pp. 56–70.
- Godefroid, P., 2020. Fuzzing: Hack, art, and science. *Commun. ACM* 63, 70–76.
- Jain, V., Rawat, S., Giuffrida, C., Bos, H., 2018. TIFF: using input type inference to improve fuzzing. In: Proceedings of the 34th Annual Computer Security Applications Conference. pp. 505–517.
- Jauernig, P., Jakobovic, D., Picek, S., Stapf, E., Sadeghi, A.-R., 2022. DARWIN: Survival of the fittest fuzzing mutators. *arXiv preprint arXiv:2210.11783*.
- Jiang, L., Yuan, H., Wu, M., Zhang, L., Zhang, Y., 2023. Evaluating and improving hybrid fuzzing. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 410–422.
- Li, Y., Ji, S., Chen, Y., Liang, S., Lee, W.-H., Chen, Y., Lyu, C., Wu, C., Beyah, R., Cheng, P., et al., 2021. UNIFUZZ: A holistic and pragmatic metrics-driven platform for evaluating fuzzers. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 2777–2794.
- Liang, J., Wang, M., Zhou, C., Wu, Z., Jiang, Y., Liu, J., Liu, Z., Sun, J., 2022. Pata: Fuzzing with path aware taint analysis. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1–17.
- Luk, C.-K., Cohn, R., Muth, R., Patil, H., Klauser, A., Lowney, G., Wallace, S., Reddi, V.J., Hazelwood, K., 2005. Pin: building customized program analysis tools with dynamic instrumentation. *Acm Sigplan Not.* 40 (6), 190–200.
- Lyu, C., Ji, S., Zhang, C., Li, Y., Lee, W.-H., Song, Y., Beyah, R., 2019. MOPT: Optimized mutation scheduling for fuzzers. In: 28th USENIX Security Symposium. USENIX Security 19, pp. 1949–1966.
- Metzman, J., Szekeres, L., Simon, L., Sprabery, R., Arya, A., 2021. Fuzzbench: an open fuzzer benchmarking platform and service. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1393–1403.
- OpenRCE, 2012. Sulley. <https://github.com/OpenRCE/sulley>.
- Peachtech, 2021. Peach: The peach fuzzer platform. <https://gitlab.com/peachtech/peach-fuzzer-community/>.
- Pham, V.-T., Böhme, M., Santosa, A.E., Căciulescu, A.R., Roychoudhury, A., 2019. Smart greybox fuzzing. *IEEE Trans. Softw. Eng.* 47 (9), 1980–1997.
- Rawat, S., Jain, V., Kumar, A., Cojocar, L., Giuffrida, C., Bos, H., 2017. VUZZer: Application-aware evolutionary fuzzing. In: NDSS, vol. 17, pp. 1–14.
- Shi, J., Wang, Z., Feng, Z., Lan, Y., Qin, S., You, W., Zou, W., Payer, M., Zhang, C., 2023. AIFORE: Smart fuzzing based on automatic input format reverse engineering. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 4967–4984.
- Su, Y., Xiong, D., Wan, Y., Shi, C., Zeng, Q., 2024. LinFuzz: Program-sensitive seed scheduling greybox fuzzing based on LinUCB algorithm. *IEEE Access*.
- You, W., Wang, X., Ma, S., Huang, J., Zhang, X., Wang, X., Liang, B., 2019. Profuzzer: On-the-fly input type probing for better zero-day vulnerability discovery. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 769–786.
- Zalewski, M., 2023. American fuzzy lop. <http://lcamtuf.coredump.cx/afl/>.
- Zhang, G., Wang, P., Yue, T., Kong, X., Huang, S., Zhou, X., Lu, K., 2024. Mobfuzz: Adaptive multi-objective optimization in gray-box fuzzing. *arXiv preprint arXiv:2401.15956*.
- Zhao, Y., Gao, L., Wan, Q., Zhao, L., 2024. Grammar-aware test case trimming for efficient hybrid fuzzing. *J. King Saud University- Comput. Inf. Sci.* 36 (1), 101920.
- Zhu, X., Wen, S., Camtepe, S., Xiang, Y., 2022. Fuzzing: a survey for roadmap. *ACM Comput. Surv.* 54 (11s), 1–36.